UNIVERSITY OF CENTRAL ARKANSAS

Kyle Urban <kurban@cub.uca.edu>

# Betterish Results!

7 messages

**Kyle Urban** <kurban@cub.uca.edu> Thu, Dec 18, 2025 at 9:39 AM
To: Zachary Stine <zstine@uca.edu>

Dear Dr Stine,

Hope all is well! After fixing some of the issues in my code and doing a bit to optimize it, I ran the first file to get a fresh version of two pickle files (one modified, one unmodified) of CLS tokens for 4244 sentence pairings (where the following selected 13 prepositions were removed: ["in", "out", "over", "under", "above", "below", "inside", "outside","around", "behind", "on" , "on top", "through"]). Following this, I plugged those two pickle files of CLS tokens into my second python file where it did the LOOCV (it ran for 12 hours) where it eventually gave me the following performance results:

For unmodified: I got a mean accuracy score of ~0.347 out of 1.0 for the classification model trying to predict either entailment, contradiction, or neutral between concatenations of CLS tokens for S1 and S2. The file showing this output score is noted as in results_noe_pcs2.txt and is attached below

For modified: I got a mean accuracy score of ~0.350 out of 1.0 for the classification model trying to predict either entailment, contradiction, or neutral between concatenations of CLS tokens for S1_mod and S2. The file showing this output score is noted in results_noe_p_cs2.txt and is attached below

Essentially, these scores are as good as the classification model randomly guessing. So the model is underfitting my data.

Keeping in mind that a 786th dimensional CLS vector concatenated (along axis =1) to another 768th dimensional CLS vector gives me a concatenated vector of 1572 dimensions, and that I used 4244 samples of sentence pairings for the test & train of the LOOCV. I am thinking that...

a. Either LOOCV might not be the most optimal classification model anymore
    -- I could try StratifiedKfold again
b. Or I need to do dimensionality reduction on the CLS vectors
c. Concatenation may not be the best representation of the representation of the CLS vectors
    -- I could try doing something different, like cosine similarity or element wise difference
d. There might be some unseen logical error in my first file that is still giving dud CLS vectors
e. a combination of multiple factors.

Option a would require the most rewriting of code, option b would consist of adding new code into old code, option c would not require more than changing 5 or 6 lines of code, but may take 12hrs of rerunning file 2 to see if there are going to be changes in the results. Option d is something I am going to check for regardless. I am going to start on option b, however, if you have suggestions on if maybe there is something I am missing, please let me know.

I thank you for taking the time this winter break to read this email and I hope that you have a wonderful day!

--
----- Kyle Urban (University of Central Arkansas Class of 2026)
----- Major: Linguistics
----- Minors: Spanish, Computer Science, Honors Interdisciplinary Studies

Honors College Freshman Mentor
Secretary-General of Model United Nations
President of Autism and Neurodiversity Alliance
President of Linguistics Club
Vice President of Social Media for Kappa Theta Pi

Phone Number: +1 (479) 231-9928 <<<< Not changing anytime soon! :)
Email: kurban@cub.uca.edu <<<< Will not be active after May 1st, 2026.
Secondary Email: kyleurban210@gmail.com <<<< Will remain active! :)

**2 attachments**

**results_noe_pcs2.txt**
1K

**results_noe_p_cs2.txt**
1K

**Kyle Urban** <kurban@cub.uca.edu> Thu, Dec 18, 2025 at 11:43 AM
To: Zachary Stine <zstine@uca.edu>

After thinking about it for a bit. I will implement Kfold cross validation before trying anything else. This is because my data set is now larger. 4244 vs 177, and thus I have a hunch LOOCV might be holding me back in terms of how long it takes to compute.
[Quoted text hidden]

**Kyle Urban** <kurban@cub.uca.edu> Thu, Dec 18, 2025 at 3:32 PM
To: Zachary Stine <zstine@uca.edu>

An update. I implemented StratifiedKfold validation for the set of 4244 for ten folds (k = 10) as well as uncommented some old code such that I would now be concatenating the CLS1 + CLS2 with the the element wise pair difference (abs(CLS1 - CLS2) and the element wise product (CLS1 * CLS2) for the the classification task (rather than just a plain concatenation of CLS1 & CLS 2). I also changed it up to give me both the f1_macro score and the accuracy. The results for the following are (also attached below in txt files results_pcs2_StratifiedKFold_evat.txt and results_p_cs2_StratifiedKFold_eval.txt respectively)...

mean accuracy of unmodified set: 0.36957606991330016
mean f1_macro of unmodified set: 0.36858942069901424

mean accuracy of mod set: 0.3635756779192031
mean f1_macro of mod set: 0.3631572225267582

The classification model is still underfitting the data. Not sure where to go from here. I think I might have an issue with how the labels are fed to the model. I will work on debugging tomorrow. In the meantime, I thank you for all your support and have a happy holiday!

[Quoted text hidden]

**2 attachments**

**results_p_cs2_StratifiedKFold_eval.txt**
1K

**results_pcs2_StratifiedKFold_eval.txt**
1K

---

**Kyle Urban** <kurban@cub.uca.edu> Sun, Dec 21, 2025 at 11:11 AM
To: Zachary Stine <zstine@uca.edu>

I have thought about it for a bit, I realize that it is also very likely (assuming I haven't messed up anything else in the code for the experimental pipeline) that CLS vector embeddings are just not informative enough for the classification model to make entailment predictions. I remember back in March of 2025, you had mentioned to me something about using "mean pooling" rather than CLS vector embeddings to do this experiment instead. I may go ahead this week and work on adding some code/reworking some code to use mean pooling instead, and see if that leads to anything different than the ~0.36ish% accuracy scores I have been getting.
[Quoted text hidden]

---

**Kyle Urban** <kurban@cub.uca.edu> Sat, Dec 27, 2025 at 12:14 PM
To: Zachary Stine <zstine@uca.edu>

Just to provide a post holiday update,

I implemented mean pooling for the set of 4244 w/stratified K fold and I got the following results:

CLS mod
Accuracy: 0.3635756779192031
Marco F1: 0.3631572225267582

CLS unmod
Accuracy: 0.36957606991330016
Marco F1: 0.36858942069901424

Mean Pool mod
Accuracy: 0.3921238009592326
Marco F1: 0.3915099859674185

Mean Pool unmod
Accuracy: 0.3947576554141302
Marco F1: 0.3943284910796713

The txt files with all the results are below in the attached, the file names also have GMT time on them.

Mean pooling of the last hidden states was slightly more informative for the classification model than the cls vector embeddings, but not by much. At this point, I can do some dimensionality reduction (I'm gonna do PCA) and see if that affects the results in any way compared to how they are now. After that, I am not sure what direction to take, but I'll probably think of something by the time the dimensionality reduction is done.

Also, using K stratified now means that it only takes 20 minutes to run the eval file, and not 12 hours like it did with LOOCV.

[Quoted text hidden]

**4 attachments**

**UNMOD_CLS_results_pcs2_StratifiedKFold_eval_2025-12-24 16_24_10.080447.txt**
1K

**MOD_MEANPOOL_results_pool_p_cs2_StratifiedKFold_eval_2025-12-27 16_26_15.607031.txt**
1K

**MOD_CLS_results_p_cs2_StratifiedKFold_eval_2025-12-27 16_18_13.257344.txt**
1K

**UNMOD_MEANPOOL_results_pool_pcs2_StratifiedKFold_eval_2025-12-27 16_22_13.287384.txt**
1K

---

**Kyle Urban** <kurban@cub.uca.edu>
To: Zachary Stine <zstine@uca.edu>

Dear Dr Stine,

I am reporting to let you know that I got sidetracked from doing dimensionality reduction. This is because I decided to clean up the code a bit (removing dead-code that I had commented out a file and putting the preparation of my data and the actual experiment of running it through the classifier into separate functions) initially in preparation to implement dim reduction and I decidec prior to testing out dimension reduction. For all prior emails, for my classification model, I have been using the liblinear solver with the L2 penalty (default). I realize that there are other solvers lbfgs, saga, sag, etc. and some google searching basically told me that liblinear may not have been the best choice for a data set of my size. Thus, I set out to implement saga as another solv functions to make them reusable). During this, google searching also told me that some solvers only work with some "penalties" (I am still confused by what this is, but I know that there are L1 something to do with Loss/Error, making them important). It wasn't hard to implement the penalty as well into the code reusability.

```
#classification_model = lg(max_iter= 1000)
#12/28/2025 update: penalty and solver now input into the function; originally, this was liblinear with no penalty
classification_model = pipe([('scaler', StandardScaler()), ('clf', lg(max_iter=5000, penalty = penalty, solver = solver, l1_ratio = l1_ratio ))]) #addi
#as of 12/18/2025, I am abandoning the LOOCV for stratified k fold since my data set is now bigger
cross_validation_method = StratifiedKFold(n_splits = 10, shuffle = True, random_state = 42)
scoring_metrics = ['accuracy', 'f1_macro'] #adding this in so I can get both!
```

Thus, I ran the experiment 5 times, (sending the results to Google Drive in text file at each incremental step so I don't lose out on the whole in case colabs crashes). Twice for liblinear (L1 and Elastic).

```
result = experiment_no_dim_reduction(X1, y1, X2, y2, X5, y5, X6, y6, "l1", "liblinear")
print(result)


experiment_no_dim_reduction(X1, y1, X2, y2, X5, y5, X6, y6, "l2", "liblinear")
experiment_no_dim_reduction(X1, y1, X2, y2, X5, y5, X6, y6, "l1", "saga")
experiment_no_dim_reduction(X1, y1, X2, y2, X5, y5, X6, y6, "l2", "saga")
experiment_no_dim_reduction(X1, y1, X2, y2, X5, y5, X6, y6, "elasticnet", "saga", l1_ratio = 0.5)
```

The results of which were the following:

- Liblinear L1:
  - CLS Concat (w/ elem diff & product)
    - unmod:
      - Accuracy: `0.3693362617598229`
      - Macro F1: `0.3682698611298111`
    - mod:
      - Accuracy: `0.3729362663715182`
      - Macro F1: `0.3719497363463188`
  - Mean Pool (w/ elem diff & product)
    - unmod:
      - Accuracy: `0.4012370872532743`
      - Macro F1: `0.40036140870889236`
    - mod:
      - Accuracy: `0.40316938756686954`
      - Macro F1: `0.4024668997635337`
- Liblinear L2:
  - CLS Concat (w/ elem diff & product)
    - unmod:
      - Accuracy: `0.36957606991330016`
      - Macro F1: `0.36858942069901424`
    - mod:
      - Accuracy: `0.3635756779192031`
      - Macro F1: `0.3631572225267582`
  - Mean Pool (w/ elem diff & product)
    - unmod:
      - Accuracy: `0.3947576554141302`
      - Macro F1: `0.3943284910796713`
    - mod:
      - Accuracy: `0.3921238009592326`
      - Macro F1: `0.3915099859674185`
- Saga L1:
  - CLS Concat (w/ elem diff & product)
    - unmod:
      - Accuracy: `0.38925129127467256`
      - Macro F1: `0.3884016731924954`
    - mod:
      - Accuracy: `0.3890172477402693`
      - Macro F1: `0.38867685522199125`
  - Mean Pool (w/ elem diff & product)
    - unmod:
      - Accuracy: `0.41659460892824196`
      - Macro F1: `0.41580057516829544`
    - mod:

        - Accuracy: `0.41155979062903525`
        - Macro F1: `0.41059512492815137`
- Saga L2:
    - CLS Concat (w/ elem diff & product)
        - unmod:
            - Accuracy: `0.3662181793027116`
            - Macro F1: `0.3649363862568981`
        - mod:
            - Accuracy: `0.37341934144991695`
            - Macro F1: `0.37286683163983747`
    - Mean Pool (w/ elem diff & product)
        - unmod:
            - Accuracy: `0.40171555063641395`
            - Macro F1: `0.40127878910039677`
        - mod:
            - Accuracy: `0.39308361003504894`
            - Macro F1: `0.392408723232648`
- Saga Elastic:
    - CLS Concat (w/ elem diff & product)
        - unmod:
            - Accuracy: `0.37269415237041137`
            - Macro F1: 0.3715714358790294
        - mod: **DNF**
    - Mean Pool (w/ elem diff & product)
        - unmod: **DNF**
        - mod: **DNF**

After about 19hrs of running, I was kicked from the Google Colab runtime for taking up too many resources, so the SAGA elastic did not finish calculating (I can just run it again separately, thc classification model, despite a change of solvers and penalties, are still just picking up on noise, and not anything real. This is perfectly okay, however it does tell me that this avenue does not model's abilities to pick apart entailment and thus I should not continue to burn time on it (and instead move to doing dimensionality reduction). Alternatively, this could also tell me that a chan significant effect on these scores, but didn't because something else is wrong (perhaps I am encoding the labels incorrectly).

**The plan from here on out:**
I will see about dimensionality reduction first before worrying about labels. If it is still low scores after dimensionality reduction, I'll see if the label encoding is messed up and rerun the prior exp not sure what other avenues could be tried (perhaps cosine similarity rather than concatenation or mean pool with the elem diff + elem product for the eval?) but at that point I think I'd have to move on writing the actual abstract/paper (to submit for the poster event at the capitol)

On labels, I currently encode them all separately:

```
y1 = label_encode(y1)
y2 = label_encode(y2)

y5 = label_encode(y5)
y6 = label_encode(y6)
```

with the function:

```
def label_encode(labels):
  print(type(labels[0]))
  le = LabelEncoder()
  labels_encoded = le.fit_transform(labels)
  print(type(labels_encoded[0]))
  return labels_encoded
```

which uses LabelEncoder from:

```
from sklearn.preprocessing import LabelEncoder
```

I should note that y1 to y6 are all the gold labels for each sentence pairing, and they should all be the same; ie: if sentence pair #117 was "entailment" in y1, then it should also be "entailment' run... y1 is used for unmodified cls concatenations, y2 for modified cls concatenations, y5 for unmodified mean pool, and y6 for modified mean pool concatenations. I am pretty sure I encode t giggles, I plug my label encoding code snippet into two LLMs: ChatGPT and Gemini, just to see what those LLMs have to spew. The former generates a claim that my code is wrong and the l I am not going to take the advice of either machine because that would be silly. However, I find it neat that two LLMs have different answers on this (rather than both giving the same-ish soun some google searching to double check that what I am doing for label encoding is in fact functional.

```
#unmodified
mean_score_pcs2_accu, all_scores_pcs2_accu, mean_score_pcs2_f1_macro, all_scores_pcs2_f1_macro = StratifiedKFold_eval(classification_model, X1, y1, scoring_metrics, cross_validation_metho
print("mean_score_pcs2_accu: ")
print(mean_score_pcs2_accu)
print("mean_score_pcs2_f1_macro:")
print(mean_score_pcs2_f1_macro)

#gimme_results(mean_score_noe_pcs2, all_scores_noe_pcs2, "results_noe_pcs2_StratifiedKFold_eval")
gimme_results_2(mean_score_pcs2_accu, all_scores_pcs2_accu, mean_score_pcs2_f1_macro, all_scores_pcs2_f1_macro, "UNMOD_CLS_results_pcs2_StratifiedKFold_eval", penalty, solver, l1_ratio)


#modified
mean_score_p_cs2_accu, all_scores_p_cs2_accu, mean_score_p_cs2_f1_macro, all_scores_p_cs2_f1_macro = StratifiedKFold_eval(classification_model, X2, y2, scoring_metrics, cross_validation_m
print("mean_score_p_cs2_accu: ")
print(mean_score_p_cs2_accu)
print("mean_score_p_cs2_f1_macro: ")
print(mean_score_p_cs2_f1_macro)


#gimme_results(mean_score_noe_p_cs2, all_scores_noe_p_cs2, "results_noe_p_cs2_StratifiedKFold_eval")
gimme_results_2(mean_score_p_cs2_accu, all_scores_p_cs2_accu, mean_score_p_cs2_f1_macro, all_scores_p_cs2_f1_macro, "MOD_CLS_results_p_cs2_StratifiedKFold_eval", penalty, solver, l1_ratio



#mean pool concatenations

#unmodified
mean_score_pool_pcs2_accu, all_scores_pool_pcs2_accu, mean_score_pool_pcs2_f1_macro, all_scores_pool_pcs2_f1_macro = StratifiedKFold_eval(classification_model, X5, y5, scoring_metrics, cr
print("mean_score_pool_pcs2_accu: ")
print(mean_score_pool_pcs2_accu)
print("mean_score_pool_pcs2_f1_macro:")
print(mean_score_pool_pcs2_f1_macro)

#gimme_results(mean_score_noe_pcs2, all_scores_noe_pcs2, "results_noe_pcs2_StratifiedKFold_eval")
gimme_results_2(mean_score_pool_pcs2_accu, all_scores_pool_pcs2_accu, mean_score_pool_pcs2_f1_macro, all_scores_pool_pcs2_f1_macro, "UNMOD_MEANPOOL_results_pool_pcs2_StratifiedKFold_eval"


#modified
mean_score_pool_p_cs2_accu, all_scores_pool_p_cs2_accu, mean_score_pool_p_cs2_f1_macro, all_scores_pool_p_cs2_f1_macro = StratifiedKFold_eval(classification_model, X6, y6, scoring_metrics
print("mean_score_pool_p_cs2_accu: ")
print(mean_score_pool_p_cs2_accu)
print("mean_score_pool_p_cs2_f1_macro:")
print(mean_score_pool_p_cs2_f1_macro)

#gimme_results(mean_score_noe_pcs2, all_scores_noe_pcs2, "results_noe_pcs2_StratifiedKFold_eval")
gimme_results_2(mean_score_pool_p_cs2_accu, all_scores_pool_p_cs2_accu, mean_score_pool_p_cs2_f1_macro, all_scores_pool_p_cs2_f1_macro, "MOD_MEANPOOL_results_pool_p_cs2_StratifiedKFold_ev

#_____#
#_____#
```

FYI: All the text files containing the scores from this trial run are below in the zip file

P.S: I thank you for all your time and support and hope the winter was spent well rested. Don't worry if you don't get to see these emails for a couple more weeks, I am doing perfectly fine (I th

[Quoted text hidden]

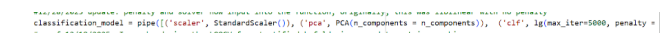
**drive-download-20251229T190821Z-1-001.zip**
13K

---

**Kyle Urban** <kurban@cub.uca.edu> Wed, Dec 31, 2025 at 1:49 PM
To: Zachary Stine <zstine@uca.edu>

Dear Dr Stine,

I hope all is well, just to let you know, I have implemented PCA dimensionality reduction into my code: I was surprised by how simple it was to implement, I thought it was much more involved than it actually was to implement and I am bummed I didn't try it first.




The following are the results comparing accuracy and marco_f1 scores for L1 liblinear logistic regression classifier using both PCA dimension reduction and no dimension reduction

With PCA dimensionality reduction (with n_components set to 0.90):

- CLS Concat
  - Unmod
    - Accuracy: `0.4626717856484043`
    - Macro_F1: `0.4601630492264232`
  - Mod
    - Accuracy: `0.45546831765356943`
    - Macro_F1: `0.45295848220445134`
- Mean Pool

    - Unmod
        - Accuracy: `0.4794750737871242`
        - Macro_F1: `0.4765031848510394`
    - Mod
        - Accuracy: `0.4818743082457112`
        - Macro_F1: `0.4789385186460865`

Without PCA dimensionality reduction:

- CLS Concat
    - Unmod
        - Accuracy:`0.3693362617598229`
        - Macro_F1: `0.3682750583268933`
    - Mod
        - Accuracy: `0.37317665098690284`
        - Macro_F1: `0.3721796966652471`
- Mean Pool
    - Unmod
        - Accuracy: `0.40147689540675147`
        - Macro_F1: `0.40062733371005477`
    - Mod
        - Accuracy: `0.4034091957203468`
        - Macro_F1: `0.4026902477795248`

Obviously, PCA dimensionality reduction was the proper step to have taken. I am going to see if the n_components have any effect on the results. The zipped files below show the results from each of these trials. The file name should note the n_components used in the PCA files, and it should say _None_ in the non PCA files.

[Quoted text hidden]

---

**drive-download-20251231T194545Z-1-001.zip**
7K